Load Testing and Capacity

Contents

Load Testing and Capacity **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 2
- Related . . . 2

Load Testing and Capacity

Scope

Measuring and defending the concurrent-user capacity of the API.

Official requirements

Subject V.7, graded:

- Be able to evaluate the load the API and backend support.
- **Justify and measure** the number of users who can use the services simultaneously.
- Named acceptable tools: Apache Bench (ab), Gatling, Siege, Tsung, JMeter.
- **Declare the server specification** — CPU, RAM, cloud or on-premise.
- The maximum user count must be consistent with the platform. Tens of users for a Raspberry Pi, thousands for a low-end server.

A number without a measurement and a declared machine does not satisfy this section.

Project policy

Deliverable

A short report committed at `docs/load/REPORT.md`, containing:

1. Machine specification: CPU model and core count, RAM, disk, network, cloud or on-premise.
2. Deployment topology: one container or several, database co-located or separate.
3. Tool and version, plus the exact command or script used — committed, so it is reproducible. The tool is **Gatling**, run from its Docker image, with simulations committed under `tests/load/` — `brain/decisions/ADR-0018-load-testing-tool.md`. It is named in the subject and supports WebSocket natively, so one methodology covers REST and SignalR fan-out.
4. Scenarios, each with its own numbers.
5. Results: requests/sec, p50 / p95 / p99 latency, error rate, at each concurrency step.
6. The bottleneck identified, with evidence.
7. The stated supported concurrent-user figure and the reasoning that connects it to the numbers.

Scenarios to measure

Scenario	Why it matters
Authenticated read of an event queue	the most frequent call during an event
Concurrent vote casting on one event	the contended write path
Playlist reorder by several collaborators	the second contended write path
SignalR fan-out: N connections in one group	realtime is the scaling risk, not REST
Login burst	the rate-limited path, must degrade correctly

Realtime fan-out is measured separately from REST throughput. Concurrent **connections** and concurrent **requests** are different limits and both must be stated.

Method

- Run against a build with production configuration, not a Debug build.
- Warm up before measuring. Discard the first run.
- Ramp concurrency in steps and record where latency degrades, not only where it fails.
- Report percentiles, never only the mean.
- An error rate above 1% means the step is over capacity — record it as the ceiling.
- Re-measure after any change to the data access or realtime layer.

Honesty rule

Report the number that was measured on the machine that was declared. An extrapolated figure is stated as an extrapolation, with its assumptions.

Checklist

- ☐ `docs/load/REPORT.md` exists with machine specification declared
- ☐ Tool, version and reproducible command committed
- ☐ REST and SignalR measured separately
- ☐ Percentiles and error rate reported per concurrency step
- ☐ Bottleneck named with evidence
- ☐ Concurrent-user figure justified from the data

References

- Gatling — https://docs.gatling.io/ (the tool this project uses)
- Apache Bench — https://httpd.apache.org/docs/current/programs/ab.html
- Apache JMeter — https://jmeter.apache.org/usermanual/
- k6 — https://grafana.com/docs/k6/latest/

Related

`../logging-observability/STANDARD.md` · `../../10-backend/signalr-realtime/STANDARD.md`